code	repo: remove HPC_instructions.md from t...	1 hour ago
.gitignore	repo: reorganize for GitHub — code/ only,...	1 hour ago
README.md	docs: redesign README with badges, hig...	1 hour ago
REPRODUCING_FIGURES.md	docs: add REPRODUCING_FIGURES.md ...	1 hour ago

README

eCPA Flash

Fast, robust phase equilibrium for CO_2 + H_2O + NaCl

python 3.11+ | platform linux | macOS | paper under review @ CEJ | T range 0–425 °C | P range 1–1500 bar

Phase stability and flash calculations for the **CO_2 + H_2O + NaCl** ternary system using the electrolyte Cubic-Plus-Association (eCPA) equation of state of Coelho, Franco & Firoozabadi (2025). Covers the full spectrum from shallow CO_2 storage aquifers to deep geothermal reservoirs.

Highlights

✅ **100% flash convergence**	across >9,800 conditions, T = 0–425 °C, P = 1–1500 bar
⚡ **3.2× faster** than cold-start SSI	via precomputed 4D solution table warm-start
🎯 **6.5% AARE** on CO_2 solubility	validated against >1,100 experimental data points
🔬 **0.33% AARE** on aqueous density	475 IAPWS-95 reference conditions
🏗️ **Reservoir simulator demo**	50×50 grid, 300 time steps, zero flash failures

What this repository provides

1. **Complete eCPA EoS** (`code/ecpa/`) — Debye–Hückel, Born, association, and permittivity terms with parameters from Coelho et al. (2025); analytical Jacobians for both Newton inner solvers.
2. **Salt-free CPA flash** (`code/CPA.py`) — Michelsen TPD stability test with six initial guesses and accelerated SSI (Jex et al., 2024). Hierarchical algorithm achieves **100% convergence** across >29,000 conditions with a **2.96×** iteration-count reduction over standard SSI.
3. **eCPA stability + flash** (`code/ecpa/flash.py`) — extension to the full ternary CO_2 + H_2O + NaCl system via warm-started K-value SSI with optional Newton polish.
4. **Precomputed solution table** (`code/results/CPA_ELV_all.parquet`) — 31T × 30P × 18z × 14m_s grid covering T = 283–728 K, P = 1–1500 bar, z = 0.05–0.90, m_s = 0–6 mol/kg.
5. **Prototype reservoir simulator** (`code/co2brine_simulator.py`) — IMPEC scheme with MFE pressure solver demonstrating production-level flash throughput.

See REPRODUCING_FIGURES.md for the complete script-by-script guide to regenerating every figure in the paper.

Requirements

```
python >= 3.11
numpy · scipy · pandas · pyarrow · matplotlib
jupyter   # optional, for the notebook
```

```
pip install numpy scipy pandas pyarrow matplotlib jupyter
```

Quick start

```
git clone https://github.com/jmoortgat/CPA_and_eCPA_flash.git
cd CPA_and_eCPA_flash/code
```

eCPA ternary flash (CO_2 + H_2O + NaCl):

```
import sys; sys.path.insert(0, '.')
import pandas as pd
from ecpa.parameters import make_params
from ecpa.guess_table  import make_guess_fn
from ecpa.flash        import flash_co2_h2o_salt_kv

params   = make_params()
guess_fn = make_guess_fn(pd.read_parquet('results/CPA_ELV_all.parquet'))

result = flash_co2_h2o_salt_kv(T=350.0, P=100.0, z=0.3, ms=1.0,
                               params=params, guess_fn=guess_fn)
print(result)
```

Salt-free CPA binary (CO_2 + H_2O):

```
import CPA
r = CPA.flash_co2_h2o_tpz(T=323.15, P_bar=100.0, z_co2=0.3)
print(r['phase'], r['x'], r['tie']['rho_mass'] * 1000, 'kg/m³')
```

Repository structure

```
CPA_and_eCPA_flash/
├── README.md
├── REPRODUCING_FIGURES.md
└── code/
    ├── ecpa/                      # eCPA package
    │   ├── flash.py               #   flash_co2_h2o_salt_kv  ← production flash
    │   ├── stability.py           #   ecpa_stability, stability_map
    │   ├── guess_table.py         #   make_guess_fn (parquet warm-start)
    │   ├── solution_table.py      #   build_solution_table
    │   ├── constants.py           #   EoS parameters & Péneloux shifts
    │   ├── parameters.py          #   make_params()
    │   ├── elv.py                 #   ELV residual + analytical Jacobian
    │   ├── flash_simplified.py    #   low-T simplified flash (T < 80 °C)
    │   ├── envelope.py            #   phase-boundary tracing
    │   ├── exp_data.py            #   experimental data loader
    │   └── ...
    ├── CPA.py                     # Salt-free CPA binary flash
    ├── co2brine_simulator.py      # IMPEC reservoir simulator
    ├── eCPA_notebook.ipynb        # Jupyter walkthrough
    ├── scripts/                   # Validation, benchmark & figure scripts
    │   ├── validate_co2h2o.py
    │   ├── validate_co2nacl_full.py
    │   ├── validate_density.py
    │   ├── run_benchmark.py
    │   ├── plot_co2h2o_figures.py
    │   ├── plot_co2nacl_figures.py
    │   └── ...
    └── results/
        └── CPA_ELV_all.parquet    # Precomputed solution table (25 MB)
```

API reference

`ecpa/flash.py`

Function	Description
`flash_co2_h2o_salt_kv(T, P, z, ms, params, guess_fn)`	**Production flash.** Hierarchical stability+flash with K-value SSI and optional Newton polish.
`flash_co2_h2o_salt_ssi(T, P, z, ms, params)`	Cold-start SSI flash (ω = 0.7). No guess table needed.

`ecpa/stability.py`

Function	Description
`ecpa_stability(T, P, z, ms, params)`	Michelsen TPD test → `(tpd_min, trial_comp, converged)`.
`stability_map(T_range, P_range, z, ms, params, n_workers)`	Parallel 2-D stability scan.

`CPA.py`

Function	Description
`flash_co2_h2o_tpz_robust(T, P_bar, z_co2)`	**100% convergence.** Stability → best-K flash → Wilson fallback.
`flash_co2_h2o_tpz(T, P_bar, z_co2, vshift_h2o, vshift_co2)`	Single-call binary flash. Returns compositions, Z-factors, `rho_mass` [kg/L].
`stability_test(T, P_bar, z, accelerated=True)`	TPD with 6 initial guesses (Jex et al., 2024).

Performance

eCPA ternary flash

Method	SSI iters	Time / call	Speedup
Cold-start SSI (Wilson init)	11.7	~10 ms	1×
Warm-start (solution table)	**3.3**	**~3 ms**	**3.2×**

Benchmark: T = 398 K, z = 0.5, m_s = 1.0 mol/kg, 30 pressure points.

Salt-free CPA binary flash

Strategy	Convergence	Mean iters
Standard SSI + Wilson K	96.0%	46.6
Accelerated SSI + Wilson K	97.1%	17.0
Hierarchical (robust)	**100%**	**12.0**

Tested at 631 experimental CO_2+H_2O points (T = 273–623 K, P = 5–3500 bar).

Validation

System	Quantity	*N*	AARE
CO_2 + H_2O (CPA)	*x*_CO_2 in water	460	8.9%
CO_2 + H_2O (eCPA)	*x*_CO_2 in water	451	8.2%
CO_2 + NaCl (eCPA)	*m*_CO_2 [mol/kg]	440	6.9%
CO_2 + NaCl (eCPA)	*x*_CO_2 [mole fraction]	99	7.0%
Aqueous density	ρ_W [kg/m³]	37	**0.33%**

Péneloux volume shifts

All shifts are in `code/ecpa/constants.py` and applied automatically via `make_params()`.

Parameter	Value	Source
`Penelouxs` (NaCl)	−53.5 cm³/mol	Coelho et al. (2025)
`Peneloux_H2O`	**+0.1105 cm³/mol**	Optimised in this work
`Peneloux_CO2`	0	Off by default

The H_2O shift is isofugacity-preserving — it improves density predictions without affecting phase compositions or VLE results.

Reproducing the figures

See [REPRODUCING_FIGURES.md] for the complete script-by-script commands to regenerate all figures in the paper and supplemental information.

Quick summary:

```
cd code
python scripts/validate_co2h2o.py        # ~5 min
python scripts/validate_co2nacl_full.py  # ~30 min
python scripts/run_parameter_scan.py     # ~20 min
python scripts/run_warmstart_scan.py     # ~27 min
python scripts/plot_co2h2o_figures.py    # Figs. 1, S1, S6
python scripts/plot_co2nacl_figures.py   # Figs. 2, 3, S2, S4
python scripts/plot_scan_figures.py      # Figs. 7, 8, 9
# ... see REPRODUCING_FIGURES.md for the full list
```

Citation

If you use this code, please cite:

```
@article{moortgat2026ecpa,
  author  = {Moortgat, Joachim and Coelho, Felipe Mour{\~a}o and Firoozabadi, Abbas},
  title   = {Fast and Robust Phase Equilibrium Computations for {CO}$_2$ + {H}$_2${O} + {NaCl}
             Mixtures Using the Electrolyte Cubic-Plus-Association Equation of State},
  journal = {Chemical Engineering Journal},
  year    = {2026},
  note    = {under review}
}
```

And the underlying eCPA parametrisation:

```
@article{coelho2025ecpa,
  author  = {Coelho, Lu{\'i}s and Franco, Luis F. M. and Firoozabadi, Abbas},
  title   = {Phase Equilibria of {CO}$_2$--Water and {CO}$_2$--Brine at High Temperatures:
             From {Monte Carlo} Simulations to the Equation of State},
  journal = {Industrial \& Engineering Chemistry Research},
  volume  = {64},
  number  = {16},
  pages   = {8492--8505},
  year    = {2025},
  doi     = {10.1021/acs.iecr.5c00134}
}
```

License

To be added upon acceptance. Code will be released under an open-source license.